

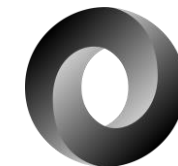


Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

APIs

Professor: Henrique Delegrego



JavaScript Object Notation (JSON)

- Formato de dados baseado em texto usado para armazenar e trocar informações
- Representa informações estruturadas como pares chave-valor, arrays ou combinações aninhadas desses elementos
- Deriva da sintaxe de objetos JavaScript, mas é apenas um formato de dados, sendo mais restrito e limitado, então não suporta funções ou métodos
- Embora tenha origem na sintaxe do JavaScript, o JSON é independente de linguagem, sendo amplamente utilizado em aplicações web, **APIs** e armazenamento de dados

```
{  
  "marca": "Honda",  
  "modelo": "Civic",  
  "ano": 2010  
}
```

APIs

No JavaScript

- No JavaScript, o JSON é usado principalmente para:
 - Serialização: Converter objetos ou arrays JavaScript em strings JSON (**stringify**)
 - Desserialização: Converter strings JSON em objetos ou arrays JavaScript (**parse**)
 - Trocar dados entre cliente e servidor (APIs), a partir de **requisições HTTP**
 - Armazenar dados estruturados em arquivos .json

APIs

Serialização (Stringify)

- Converte objetos ou arrays JavaScript em strings **JSON**

```
// Exemplo de uso do JSON.stringify
const objeto = {marca: "Honda", modelo: "Civic", ano: 2010 }; // Objeto JavaScript
const jsonString = JSON.stringify(objeto); // Converte o objeto JavaScript em uma string JSON
console.log(stringJson); // Exibe a string JSON resultante
```

APIs

Desserialização (Parse)

- Converte uma string **JSON** em um objeto ou array JavaScript

```
// Exemplo de uso do JSON.parse
const string = '{"marca": "Honda", "modelo": "Civic", "ano": 2010}'; // String JSON
const obj = JSON.parse(string); // Converte a string JSON em um objeto JavaScript
console.log(obj.marca); // Acessa a propriedade "marca" do objeto
console.log(obj.modelo); // Acessa a propriedade "modelo" do objeto
console.log(obj.ano); // Acessa a propriedade "ano" do objeto
```

APIs

Introdução a APIs

- Conjunto de regras e protocolos que permitem que diferentes sistemas de software se comuniquem entre si de maneira estruturada e previsível
- Especifica um conjunto de **endpoints** que um programa pode usar para solicitar dados ou funcionalidades de outro sem precisar entender sua implementação interna
- **Podem ser categorizadas por:**
 - Escopo de acesso (públicas, parceiras, privadas)
 - Estilo arquitetural (REST, SOAP, GraphQL, gRPC)
 - Caso de uso (APIs de dados, APIs de SO, APIs de hardware)
- APIs web utilizam o protocolo **HTTP**

APIs

REST API

- Introduzido em 2000 por Roy Fielding, é a API web mais utilizada
- Conjunto de regras e convenções para construir e interagir com serviços web fazendo com que o cliente e o servidor sejam independentes
- É **stateless** (sem estado): Cada requisição do cliente deve conter todas as informações necessárias para o servidor processá-la. O servidor não armazena estado de sessão
- Uma requisição a uma API REST é uma chamada **HTTP** enviada por um **cliente** a um **servidor**, especificando um recurso por meio de uma **URL**, um **método HTTP**, **cabeçalhos** e, em certas ocasiões, um corpo contendo dados (normalmente JSON)
- O servidor então processa essa requisição e retorna uma resposta composta por um **código de status**, cabeçalhos e um corpo opcional com os dados ou resultado solicitados, geralmente formatado em JSON

APIs

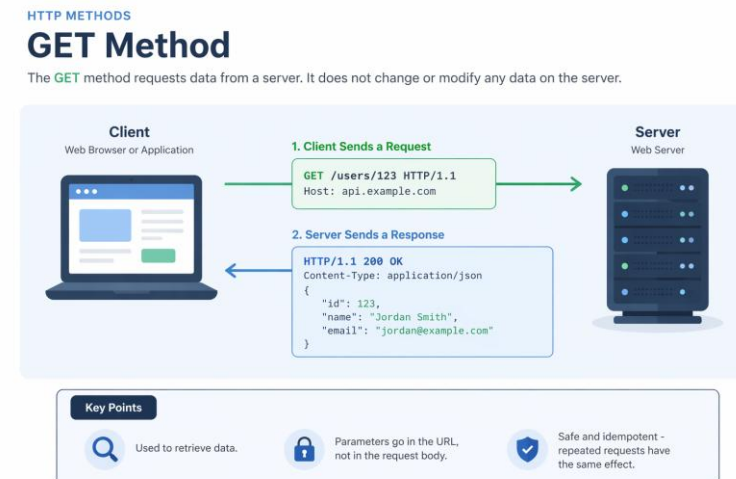
Métodos HTTP

- Métodos de requisição suportados pelo protocolo HTTP
- Cada método indica um tipo específico de operação a ser realizada
- **São eles:**
 - GET
 - POST
 - PUT
 - DELETE

APIs

GET

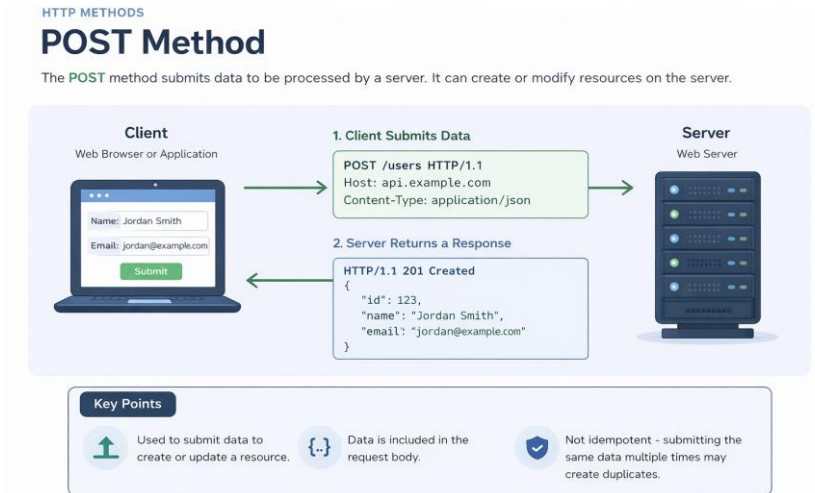
- Recupera dados de um servidor no recurso especificado
- **Exemplo:**
 - Buscar um perfil de usuário
 - Carregar uma lista
 - Carregar uma página web ou uma imagem
- Não modifica nenhum dado no servidor
- Múltiplas requisições idênticas geram o mesmo resultado (idempotente)
- Não possui corpo de requisição
- Retorna os dados solicitados, como HTML, JSON, XML, imagens ou arquivos



APIs

POST

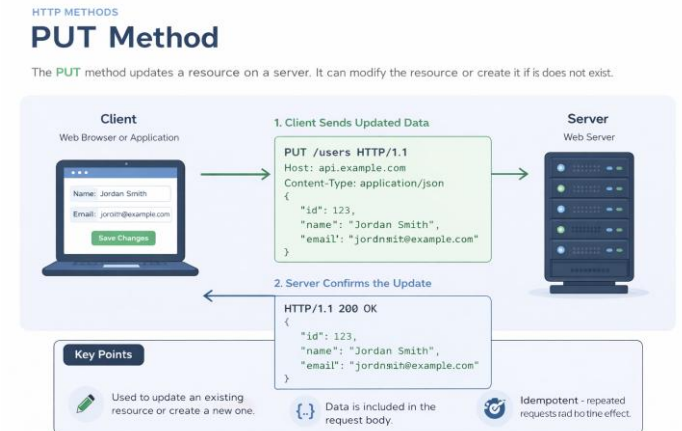
- Envia dados para um servidor no recurso especificado
- **Exemplo:**
 - Criar um novo usuário
 - Publicar algo em uma rede social
 - Enviar um formulário
- Possui corpo de requisição no formato de JSON, XML, formulário ou arquivos
- Retorna o resultado da operação, como confirmação, dados criados, mensagens ou mensagens de erros



APIs

PUT

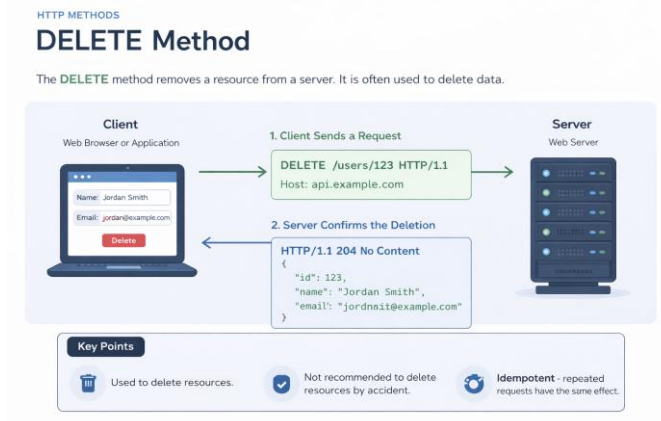
- Atualiza ou substitui dados em um servidor no recurso especificado
- **Exemplo:**
 - Atualizar número de contato de um usuário
- Possui corpo de requisição no formato de JSON, XML, formulário ou arquivos
- Retorna o resultado da atualização, como confirmação, dados atualizados ou mensagens de erro
- Se o recurso não existir, alguns servidores podem criá-lo



APIs

DELETE

- Remove dados de um servidor no recurso especificado
- **Exemplo:**
 - Deletar um usuário
- Geralmente não possui corpo de requisição
- Retorna o resultado da operação, como confirmação da exclusão, status ou mensagens de erro



APIs

Status Codes

- Códigos de três dígitos padronizados que os servidores enviam em resposta a uma requisição HTTP de um cliente, indicando o resultado da requisição
- Agrupados em cinco classes, cada uma começando com um dígito diferente (1xx a 5xx)
 - 1xx - Informativo: Requisição recebida, processo em continuidade
 - 2xx - Sucesso: A ação foi recebida, compreendida e aceita com sucesso
 - 3xx - Redirecionamento: Ação adicional é necessária para completar a requisição
 - 4xx - Erro do cliente: A requisição contém sintaxe incorreta ou não pode ser atendida
 - 5xx - Erro do servidor: O servidor falhou em atender uma requisição

APIs

2xx (Sucesso)

- **200 (OK):** A requisição foi bem-sucedida, e a resposta contém os dados solicitados. Código de sucesso genérico
- **201 (Created):** A requisição resultou na criação de um novo recurso. Geralmente utilizado com requisições POST
- **204 (No Content):** A requisição foi bem-sucedida, mas não há conteúdo para retornar. Geralmente usado para atualizações ou exclusões

APIs

4xx (Erro do Cliente)

- **400 (Bad Request):** O servidor não pode processar a requisição devido a um erro do cliente. Exemplo: Um JSON malformatado em uma requisição POST
- **401 (Unauthorized):** A requisição exige autenticação, mas o cliente ainda não forneceu credenciais (tokens) válidas
- **403 (Forbidden):** Cliente autenticado, mas sem permissão para acessar o recurso
-  **404 (Not Found):** O servidor não consegue encontrar o recurso solicitado. Apenas indica que o recurso está ausente, não se a ausência é temporária ou permanente
- **409 (Conflict):** A requisição é reconhecida, mas não pôde ser concluída devido a uma violação do estado atual do recurso. Exemplo: email duplicado, versão desatualizada, etc
- **429 (Too Many Requests):** Cliente enviou muitas requisições ao servidor em um curto período de tempo, ultrapassando os limites estabelecidos pelo servidor

APIs

5xx (Erro do servidor)

- **500 (Internal Server Error):** Erro genérico indicando que o servidor encontrou uma condição inesperada
- **503 (Service Unavailable):** O servidor está temporariamente indisponível, muitas vezes devido a manutenção ou sobrecarga

Agora:

- Vamos conhecer o Thunder Client, ViaCEP e JSONPlaceholder
- Fazer o exercício de API

APIs

Fetch

- Interface assíncrona baseada em **Promises**, usada para fazer requisições HTTP
- Separa o processamento de requisições e respostas em etapas distintas:
 - **fetch(url, options)** retorna uma **Promise** que se resolve com um objeto **Response**
 - Processo de conversão da resposta usando **.json()**

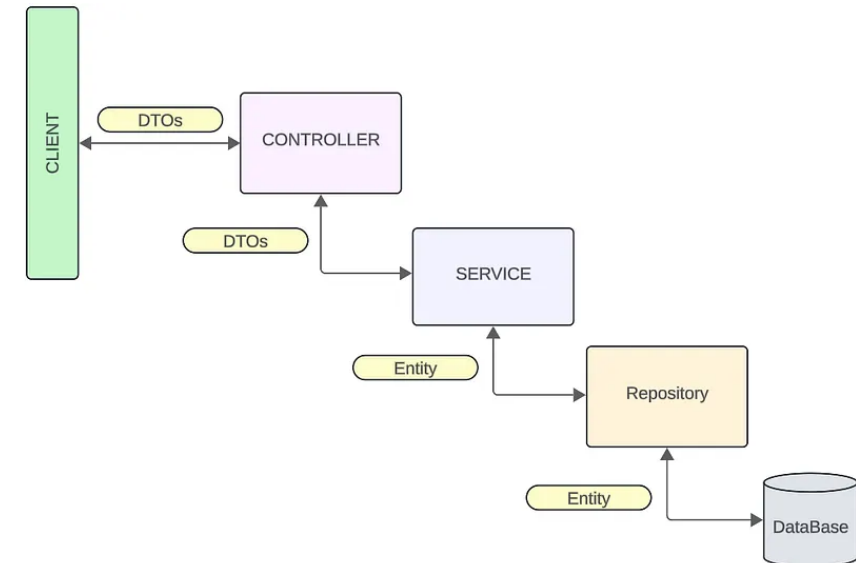
```
29 async function fetchPosts() {
30
31     // Realiza a requisição HTTP GET usando fetch e aguarda a resposta
32     const response = await fetch('https://jsonplaceholder.typicode.com/posts');
33
34     // Verifica se a resposta HTTP foi bem-sucedida (status 200-299)
35     // Caso contrário, lança um erro
36     if (!response.ok) {
37         throw new Error("Erro ao buscar posts");
38     }
39
40     // Converte o corpo da resposta para JSON
41     const data = await response.json();
42
43     return data;
44 }
```

```
3 async function criarPost(dados) {
4     try {
5         // Faz uma requisição HTTP POST para a API
6         const response = await fetch('https://jsonplaceholder.typicode.com/posts', {
7             method: 'POST', // Define o método HTTP como POST (envio de dados)
8             headers: {
9                 'Content-Type': 'application/json',
10            },
11            body: JSON.stringify(dados) // Converte o objeto JS para JSON
12        });
13
14        if (!response.ok) {
15            throw new Error(`Erro HTTP! status: ${response.status}`);
16        }
17
18        const data = await response.json();
19
20        return data;
21    } catch (erro) {
22        console.error("Erro ao criar post:", erro);
23    }
24 }
25 }
```

APIs

Arquitetura Web

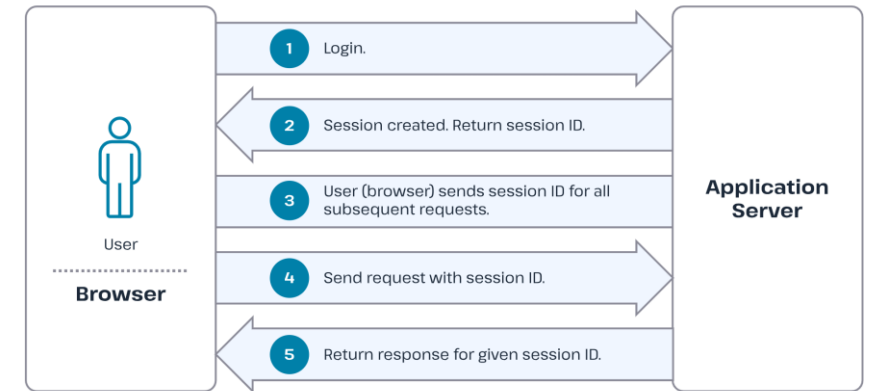
- Se refere ao design, à estrutura e à organização de como as aplicações web, os websites e os serviços web funcionam, se comunicam e interagem entre eles
- **Componentes da arquitetura de Três Camadas:**
 - **Client (Front-End):** É o que os usuários veem e com o que interagem, a interface do usuário (UI). Envia pedidos ao servidor seguindo o protocolo HTTP e exibe as respostas
 - **Server (Back-End):** Lida com a regra de negócio, processa pedidos dos clientes e se comunica com bancos de dados
 - **Database (Data Layer):** Armazena os dados usados pela aplicação



APIs

HTTP Cookie

- Dados transmitidos por cabeçalhos HTTP que um servidor solicita ao navegador para armazenar e enviar de volta
- Sem cookies, cada requisição e estado seriam independentes e não persistiriam entre carregamentos de página
- Quando um usuário faz login em um site, o servidor normalmente gera um **identificador de sessão** único e o armazena em um cookie, enviando-o de volta em solicitações subsequentes, evitando o armazenamento de dados confidenciais no navegador





Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

0800 048 1212     **sc.senai.br**

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC